

Part A: Min-Max Continual-RL Consolidation

Constrained two-policy consolidation, experts NOT stored (config ppo.method = constrained).

Overview

Tasks $1, \dots, K$ arrive in sequence. A single shared-trunk, multi-head GLOBAL policy $\pi_G = \pi_\phi$ must keep performing on every seen task. Nothing about the past is stored except the current parameters ϕ : past-task quantities are re-estimated by FRESH rollouts in the old environments each iteration (replay-free). NO per-task expert model is saved; the current task's floor is supplied on the fly by a throwaway LOCAL policy trained from scratch.

Each stage $k \geq 2$ runs a three-checkpoint local-global cycle (eq 4):

$\pi_G^- \rightarrow \pi_L \rightarrow \pi_G^+$. Here $\pi_G^- = \pi_{\phi^-}$ is the previous global (held FIXED all stage); π_L is a local specialist trained by plain PPO on task k from

$\theta^{(0)} = \phi^-$ (eqs 5, 19), then FROZEN, whose value $\hat{V}_k^L$ is the current-task

floor; and $\pi_G^+ = \pi_{\phi^*}$ is the consolidated global, produced by maximizing the

weighted past objective $\sum_{i < k} \omega_i V_i^\phi$ subject to the one-sided current-task floor, enforced as a primal-dual saddle over (ϕ, μ) . At stage end

$\phi^- \leftarrow \phi^*$ and task k joins the past set (eq 6). Only the ACTOR carries the

constraint; the critic, GAE, value-MSE and entropy are ordinary shared-trunk PPO (Algorithm 1).

Top level and stage cycle

Algorithm 1: run (method = constrained)

Input: ordered family of K tasks (Atari games), each with a greedy threshold

Input: shared-trunk multi-head global policy π_ϕ

Input: budgets task1_iters, local_iters, global_iters; cycles_per_task (usually 1)

Input: dual controller (projected ascent): μ init, step η , ceiling $\mu_{\max}$; tolerance ε

Output: final global policy $\rightarrow$ final_policy.pt; per-task checkpoints global_after_taskk.pt, local_after_

Output: lower-triangular greedy-score forgetting matrix $\rightarrow$ eval_matrix.json

- 1 STAGE 1 (plain PPO on the global net): LocalTrainer.train(π_ϕ , task1, task1_iters)
- 2 record its greedy score as task 1's local reference (no local phase, no floor, no μ)

```

3 row1 ← evaluate-row(1); append to eval-matrix; save progress

4 for k = 2, ..., K do
5     constrained-task(k)
6     rowk ← evaluate-row(k); append to eval-matrix; save progress

    end for
7 save eval-matrix and final_policy =  $\phi^*$ 

```

Algorithm 2: constrained-task(k) (one local-global cycle)

Input: task k ; past tasks $\{1, \dots, k-1\}$; global policy π_ϕ (parameters ϕ^- carried in)

Output: consolidated global π_{ϕ^+} (becomes π_{ϕ^-} of stage $k+1$, eq 6)

```

1 for cycle = 1, ..., cycles_per_task do
2     LOCAL PHASE (produce the frozen floor  $\hat{V}_k^L$ ; eqs 5, 18-23):
3          $\pi_L \leftarrow \text{clone}(\pi_\phi, \text{trainable})$  ( $\theta^{(0)} = \phi^-$ )

4         LocalTrainer.train( $\pi_L$ , taskk, local_iters) (plain PPO); then FREEZE  $\bar{\pi}_L$ 
5         save  $\bar{\pi}_L \rightarrow \text{local\_after\_taskk.pt}$  ( $\bar{\pi}_L$  IS Part A's per-task reference)
6          $\hat{V}_k^L \leftarrow \text{eval-value}(\bar{\pi}_L, \text{task}_k)$  (on-policy STOCHASTIC MC return; the floor)

7     GLOBAL PHASE (constrained consolidation; eqs 25-48):
8          $\mu\text{-controller.reset}()$  (fresh dual per phase:  $\mu \leftarrow \text{init}$  unless warm-started)

9          $\omega \leftarrow [1/k]$  repeated  $k-1$  times (uniform past weights  $\omega_i = 1/k$ )

10        GlobalTrainer.train( $\pi_\phi$ , taskk, past,  $\hat{V}_k^L$ , dual,  $\omega$ ,  $\varepsilon$ , global_iters)
11        ( $\bar{\pi}_L$  passed only for KL-gap logging / optional BC term)
12    log the current-task reference  $\hat{V}_k^L$ 
    end for

```

Local player: plain PPO on one task

The local player is standard PPO with a single rollout stream and actor coefficient 1 (eqs 18-23); the coeff-sum normalization inside Algorithm 1 is therefore a no-op here. It early-stops when the greedy game score meets the task threshold patience times.

Algorithm 3: LocalTrainer.train(π , task, num_iters)

Input: policy π ; task; iteration cap; greedy threshold τ_{thr}

Output: trained π (best-greedy snapshot kept if a late phase diverges)

```
1 opt  $\leftarrow$  Adam; collector  $\leftarrow$  one rollout stream on task

2 for it = 1, ..., num_iters do
3   batch  $\leftarrow$  collect( $\pi$ ) with GAE

4   optimize-batches( $\pi$ , opt, [batch], [1]) (Algorithm 1)
5   periodically: greedy score; EARLY STOP once score  $\geq \tau_{\text{thr}}$  for patience checks
   end for
```

Global player: constrained consolidation

The global player maximizes $\sum \omega_i V_i^\phi$ (eq 25) while a single multiplier μ enforces the floor $V_k^G \geq \hat{V}_k^L$ through the one-sided squared hinge $F_k = [\hat{V}_k^L - V_k^G]^2 + \epsilon$ (eqs 26, 27). The current task's actor coefficient is the differentiated hinge $2\mu[\hat{V}_k^L - \hat{V}_k^G]_+$ (eqs 38, 40). The value $\hat{V}_k^G$ is a FRESH on-policy Monte-Carlo estimate (never read off the moving critic head), refreshed on a SLOW dual timescale.

Algorithm 4: GlobalTrainer.train(π_ϕ , task_k, past, $\hat{V}_k^L$, dual, ω , ϵ , num_iters)

Input: global π_ϕ ; frozen floor $\hat{V}_k^L$; past weights $\omega_i = 1/k$; tolerance ϵ

Input: config: constraint_every (dual timescale), constraint_episodes, past_task_sampling

Output: consolidated $\pi_\phi +$

```

1  opt  $\leftarrow$  Adam; one persistent replay-free collector per task (all past + current)

2   $\mu \leftarrow$  dual.value;  $s_k \leftarrow 0$ ;  $F_k \leftarrow 0$ ;  $\hat{V}_k^G \leftarrow \text{NaN}$ 

3  for it = 1, ..., num_iters do
4      (1) FRESH rollouts, past + current, this iteration (replay-free):
5          if past-sampling = sample: pick one random past task  $j$ ; roll only it,
              with coeff  $\omega_j \cdot n_{\text{past}}$  (unbiased estimate of the full past sum)
6          else: roll ALL past tasks  $i < k$ , each with coeff  $\omega_i = 1/k$ 
7          cur-batch  $\leftarrow$  collect(task  $k$ )

8      (2) SLOW dual timescale: every constraint_every iterations do:
9           $\hat{V}_k^G \leftarrow$  fresh on-policy Monte-Carlo value of  $\pi_\phi$  on task  $k$  (eq 42)

10          $s_k \leftarrow \max(0, \hat{V}_k^L - \hat{V}_k^G) \quad ([\hat{V}_k^L - \hat{V}_k^G]_+)$ ;  $F_k \leftarrow s_k^2$  (eq 26)

11          $\mu \leftarrow$  dual.update( $F_k, \varepsilon$ ) (projected dual ascent, eq 47)

12         coeff $_k \leftarrow \mu \cdot 2 \cdot s_k$  (differentiated hinge, eqs 38, 40; held fixed between refreshes)

13     (3) one PPO update over ALL streams (past first, current last):
14         streams  $\leftarrow$  past-batches + [cur-batch]; coeffs  $\leftarrow$  past-coeffs + [coeff $_k$ ]

15         optimize-batches( $\pi_\phi$ , opt, streams, coeffs, kl_stop = false) (Algorithm 1)
16     early-stop check (default: current game hits its greedy threshold; V5: every
        seen task  $\geq$  retention frac of its local reference)
    end for
17 return  $\pi_{\phi^+} \leftarrow \pi_\phi$ 

```

Dual update and forgetting row

Algorithm 5: ProjectedAscentDual.update(F_k, ε) (eq 47)

Input: current-task squared shortfall F_k ; tolerance ε ; step η ; ceiling $\mu_{\max}$

Output: updated multiplier μ

1 $\mu \leftarrow \text{clip}(\mu + \eta(F_k - \varepsilon), 0, \mu_{\max})$; **return** μ

lower clip at 0 is the $[\cdot]_+$ projection of eq 47; the upper cap $\mu_{\max}$ is a magnitude-only deviation

Algorithm 6: evaluate-row(k) (post-task greedy forgetting row)

Input: global π_ϕ ; seen tasks $1, \dots, k$ (or all K if eval-all-tasks)

Output: row $[\text{score}_1, \dots, \text{score}_k]$ of the forgetting matrix (+ per-game std)

1 **for each** seen task i : score $\leftarrow$ reported GREEDY (argmax) game SCORE of π_ϕ

pooled over fixed-seed episodes (exact greedy via no-op enumeration if enabled)

reported metric is task PERFORMANCE (raw score), NOT the discounted value V

Faithful practical deviations from the raw equations

D1 (coeff-sum normalization, Algorithm 1): every actor coefficient is divided by their sum before the PPO step. This is a direction-preserving positive rescaling that keeps the shared critic from starving under a large μ ; the saddle fixed point of eq 29 is unchanged. D2 (μ ceiling): eq 47 projects only onto $\mu \geq 0$; the code also caps μ at $\mu_{\max}$.

Both deviations are magnitude-only and leave the update direction and the saddle point unchanged. VERIFICATION: the Part A derivation is math-verifier SOUND and the implementation PASSED code-verifier against eqs 26 / 38 / 40 / 47.